

Chapter 9: Three-Dimensional Viewing and Surface

0. Translation, Rotation, and Scaling in Three-Dimensional Transformations

Three-dimensional transformations are geometric operations used to change the position, orientation, and size of objects in a 3D coordinate system. They are represented using 4×4 homogeneous transformation matrices, allowing multiple transformations to be combined into one matrix.

A. Translation

Translation moves an object from one location to another without changing its size, shape, or orientation. If a point is

$$P(x, y, z)$$

after translation by (T_x, T_y, T_z) , the new point becomes

$$x' = x + T_x$$

$$y' = y + T_y$$

$$z' = z + T_z$$

Translation Matrix

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & T_x \\ 0 & 1 & 0 & T_y \\ 0 & 0 & 1 & T_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

B. Rotation

Rotation changes the **orientation** of an object about a specified axis while preserving its size and shape. In 3D graphics, rotation can occur about the **X-, Y-, or Z-axis**, or about any arbitrary axis. Positive rotation follows the right-hand rule.

Rotation about Z-axis

$$x' = x \cos \theta - y \sin \theta$$

$$y' = x \sin \theta + y \cos \theta$$

$$z' = z$$

Matrix

$$\begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotation about X-axis

$$x' = x$$

$$y' = y \cos \theta - z \sin \theta$$

$$z' = y \sin \theta + z \cos \theta$$

Matrix:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotation about Y-axis

$$x' = x \cos \theta + z \sin \theta$$

$$y' = y$$

$$z' = -x \sin \theta + z \cos \theta$$

Matrix

$$\begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

C. Scaling

Scaling changes the **size** of an object. The shape remains the same if the scaling factors are equal in every direction.

- Enlargement if scaling factor > 1
- Reduction if scaling factor < 1

For scaling factors (S_x, S_y, S_z) :

$$x' = S_x x$$

$$y' = S_y y$$

$$z' = S_z z$$

Scaling Matrix

$$\begin{bmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Types

- Uniform Scaling
- Differential (Non-uniform) Scaling

1. Three-Dimensional Display Methods

Definition

A Three-Dimensional (3D) Display Method is the process of displaying a 3D object on a 2D computer screen. Since monitors are two-dimensional, the computer converts the 3D scene into a 2D image using viewing and projection techniques.

Why is 3D Display Needed?

Unlike 2D graphics, 3D graphics must consider:

- Height (X-axis)
- Width (Y-axis)
- Depth (Z-axis)

To display a realistic image, the computer must:

- Position the camera (viewer)
- Select the viewing direction
- Project the 3D object onto a 2D plane

- Identify visible surfaces
- Hide invisible surfaces
- Apply lighting and shading if needed

Process of Displaying a 3D Object

1. Create the object in world coordinates.
2. Set the camera (viewpoint).
3. Transform the object into viewing coordinates.
4. Project the object onto the display plane.
5. Remove hidden surfaces.
6. Apply shading and lighting.
7. Display the final image.

Types of 3D Display

1. Wireframe Display

- Shows only edges and vertices.
- Looks like a skeleton.
- Fast to draw.
- Hidden surfaces are visible.
- Mainly used during object design.

Advantages

- Simple
- Requires less memory
- Fast rendering

2. Surface Rendering

- Displays only the visible surfaces.
- Hidden surfaces are removed.
- Supports color, texture and lighting.
- Produces realistic images.

Difference Between Wireframe and Surface Rendering

Wireframe	Surface Rendering
Shows edges only	Shows complete surfaces
Hidden lines visible	Hidden lines removed
Fast	Slower
Less realistic	More realistic

3D Object Representation

Definition

3D object representation means the method used to store and describe the shape of a 3-dimensional object inside a computer so that it can be displayed, edited, and manipulated. Since real objects can be simple (a box) or complex (a human face, a cloud, a mountain), computer graphics uses several

different schemes to represent them, depending on how much accuracy, memory, and rendering speed is required.

Broadly, all representation schemes fall into two big groups:

- Boundary Representations (B-reps) — describe a 3D object by specifying its bounding surface.
- Space-Partitioning Representations — describe the interior properties of an object by partitioning the spatial region containing the object into a set of small, non-overlapping, contiguous solid cells (usually cubes), e.g., octrees.

Choosing a representation depends on the application: polygon meshes are fast for real-time rendering (games), spline surfaces give smooth curved shapes (car bodies, animation characters), and octrees are useful for volume data (medical scans) and space-related calculations like collision detection.

Categories of 3D Object Representation

1. Polygon Surface Representation

One of the most common methods of representing a 3D object is Polygon Surface Representation. In this method, an object is made up of many flat polygons (usually triangles or quadrilaterals). The object is defined by the vertices of these polygons. During transformations, the computer updates the vertices and redraws the polygon surfaces.

Example:

A cube is represented using 6 square faces. When the cube is moved, the coordinates of all its vertices are updated.

2. Hierarchical Representation

Hierarchical Representation is used to build a complex object from several smaller parts. Each part is created separately and then connected to form the complete object. When the main object moves or rotates, all connected parts move together.

Example:

A human body is modeled hierarchically, where the arms, legs, and head are connected to the body.

3. Constructive Solid Geometry (CSG)

Constructive Solid Geometry (CSG) creates complex objects by combining simple 3D shapes such as cubes, spheres, and cylinders. It uses three Boolean operations:

- Union – Combines objects
- Intersection – Keeps only the common part
- Difference – Subtracts one object from another

Example:

A **table lamp** can be created by combining a cylinder, a cone, and a sphere.

4. Octree Representation

Octree Representation divides a 3D space into eight equal cube-shaped regions (octants). Regions containing part of the object are divided further into smaller cubes. This method saves memory and speeds up graphics operations.

Example:

In a 3D game, an octree helps render only nearby objects instead of the entire scene.

5. Polyhedron Representation

A Polyhedron is a solid object made entirely of flat polygon faces connected by edges and vertices. It is widely used because it is simple to store and render.

Example:

A cube is a polyhedron with 6 faces, 12 edges, and 8 vertices.

6. Curved Surface Representation

Curved Surface Representation uses mathematical equations instead of flat polygons to create smooth surfaces. Bézier and B-Spline curves are common examples.

Example:

The smooth body of a car or an airplane wing is modeled using curved surfaces.

7. Specialized Modeling Representations

Specialized Modeling Representations are designed for specific applications such as engineering, medicine, and chemistry.

Example:

A CAD model is used to design machine parts, while a ball-and-stick model is used to represent molecules.

2. Parallel Projection

Projection is the process of converting a 3D object into a 2D image on the screen or a viewing plane.

Definition

Parallel Projection is a projection method in which all projection lines remain parallel while projecting the object onto the projection plane.

Characteristics

- Projection lines never meet.
- Parallel lines remain parallel.
- Object size remains constant.
- No perspective effect.
- Accurate measurements possible.

Types of Parallel Projection

Orthographic Projection

Projection lines are perpendicular to the projection plane.

Produces:

- Front View
- Top View
- Side View

Oblique Projection

- The projection lines are inclined (not perpendicular) to the view plane.

General Parallel Projection Transformation

A General Parallel Projection Transformation is a method of projecting a 3D object onto a 2D view plane using parallel projection lines. Since the projection lines never meet, the size of the object does not change with distance. It is mainly used in engineering drawings and CAD.

There are two types of parallel projection:

- **Orthographic Projection:** The projection lines are perpendicular to the view plane.
- **Oblique Projection:** The projection lines are inclined (not perpendicular) to the view plane.

1. Shearing Operation

For an oblique projection, a shearing transformation is applied. It shifts the x and y coordinates according to the z value, while the z-coordinate remains unchanged. This converts the oblique view into a regular parallelepiped, making clipping easier.

2. Projection Vector

The direction of projection is represented by the **projection vector**

$$V_p = (p_x, p_y, p_z)$$

The shear parameters are calculated as:

$$a = -\frac{p_x}{p_z}$$
$$b = -\frac{p_y}{p_z}$$

3. General Parallel Projection Matrix

The transformation matrix is

$$M = \begin{bmatrix} 1 & 0 & -\frac{p_x}{p_z} & 0 \\ 0 & 1 & -\frac{p_y}{p_z} & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

For orthographic projection, $p_x = p_y = 0$, so the matrix becomes the identity matrix (no shearing is required).

4. View Volume and Clipping

After applying the projection, the view volume becomes a regular parallelepiped, making clipping calculations simpler. Front (near) and back (far) clipping planes are used to limit the visible part of the scene.

Example: An engineer drawing the front and side views of a machine part uses parallel projection. Even if one part is farther away, its size remains the same because the projection lines are parallel.

3. Perspective Projection

Perspective Projection is a projection technique where all projection lines meet at a single point called the Center of Projection (COP). Objects farther away appear smaller, making the image look realistic.

Perspective Projection Matrix

A Perspective Projection Matrix is used to convert a 3D object into a 2D image. In this projection, all projection lines meet at a single point called the Projection Reference Point (PRP) or Center of

Projection. This makes near objects appear larger and distant objects appear smaller, creating a realistic view like human vision or a camera.

Perspective Projection Matrix

$$M_{persp} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -\frac{z_{vp}}{d_p} & \frac{z_{vp}z_{prp}}{d_p} \\ 0 & 0 & -\frac{1}{d_p} & \frac{z_{prp}}{d_p} \end{bmatrix}$$

where,

- z_{prp} = Projection Reference Point (PRP)
- z_{vp} = View Plane
- $d_p = z_{prp} - z_{vp}$

Features

- Produces a realistic 3D view.
- Distant objects appear smaller.
- Parallel lines seem to meet at a vanishing point.
- Commonly used in games, movies, and 3D graphics.

Example

A railway track appears to meet at the horizon even though the rails are parallel. This is an example of perspective projection.

Equations of the Perspective Projection Matrix (Explanation)

The perspective projection equations describe how a 3D point is projected onto a 2D view plane. They are based on the idea that every projection line passes through the Projection Reference Point (PRP).

Step 1: Projection Line Equations

A point on the projection line is given by

$$\begin{aligned} x' &= x - xu \\ y' &= y - yu \\ z' &= z - (z - z_{prp})u \end{aligned}$$

where u is a parameter that represents a point on the projection line.

Step 2: Find the Value of u

Since the projected point lies on the view plane,

$$z' = z_{vp}$$

Therefore,

$$u = \frac{z_{vp} - z}{z_{prp} - z}$$

Step 3: Final Projection Equations

Substituting the value of u gives the final coordinates on the view plane:

$$\begin{aligned} x_p &= x \left(\frac{d_p}{z_{prp} - z} \right) \\ y_p &= y \left(\frac{d_p}{z_{prp} - z} \right) \end{aligned}$$

where

$$d_p = z_{prp} - z_{vp}$$

Simple Explanation

- If an object is **near the viewer**, it appears **larger**.
- If an object is **far from the viewer**, it appears **smaller**.
- This creates the realistic depth effect seen in perspective projection.

Example

Suppose

- $x = 6$
- $y = 4$
- $z = 10$
- $d = 5$

Using the simplified equations,

$$x_p = \frac{dx}{z}$$
$$y_p = \frac{dy}{z}$$

we get

$$x_p = \frac{5 \times 6}{10} = 3$$
$$y_p = \frac{5 \times 4}{10} = 2$$

So, the projected point is

$$(3, 2)$$

Note: The perspective projection matrix (Topic 14) provides the mathematical transformation in matrix form, while the equations (Topic 15) explain how the final projected coordinates (x_p, y_p) are calculated from a 3D point.

4. Depth Queuing (Depth Cueing)

Depth Queuing (Depth Cueing) is a technique used in computer graphics to create a sense of depth in a 3D scene. It works by changing the brightness, intensity, or color of objects according to their distance from the viewer. Objects that are closer appear brighter, while objects that are farther away appear dimmer or darker. This helps the viewer easily understand which objects are near and which are far.

Need for Depth Queuing

In a **wireframe model**, all edges are drawn with the same brightness, making it difficult to identify which parts of the object are closer or farther from the viewer. As a result, the object may look confusing. Depth queuing solves this problem by gradually reducing the brightness or changing the color of distant objects. This gives the image a more realistic three-dimensional appearance and improves depth perception.

Working of Depth Queuing

Depth queuing works in the following steps:

Step 1: Calculate the distance of every object or line from the viewer.

Step 2: Assign the highest brightness (or strongest color) to the objects that are closest to the viewer.

Step 3: Gradually reduce the brightness or intensity as the distance increases.

Step 4: Display the scene so that near objects appear bright and far objects appear dim or faded.

As a result, the viewer can easily recognize the depth of the scene.

5. Surface Rendering

Surface rendering is used to make a 3D object look realistic. It displays the visible surfaces of an object by applying lighting, shading, color, and surface properties. Hidden surfaces are not shown.

A. Steps of the Depth-Buffer Algorithm (Z-Buffer Algorithm)

The Depth-Buffer Algorithm (or Z-Buffer Algorithm) is a hidden surface removal technique used to determine which surface is visible at each pixel. It uses two buffers:

- Depth Buffer (Z-buffer): Stores the depth (z-value) of the nearest surface.
- Refresh Buffer (Frame Buffer): Stores the color or intensity of the visible surface.

Steps of the Algorithm

Step 1: Initialize the Buffers

- Set all values in the depth buffer to 0 (or the farthest depth).
- Set all pixels in the refresh buffer to the background color.

Step 2: Process Each Surface

Process every polygon (surface) in the scene one by one.

Step 3: Calculate the Depth

For each pixel (x, y) on the polygon, calculate its depth (z-value).

Step 4: Compare the Depth

Compare the new z-value with the value already stored in the depth buffer.

- If the new surface is closer to the viewer, continue to the next step.
- Otherwise, ignore the pixel.

Step 5: Update the Buffers

If the new surface is closer:

- Update the depth buffer with the new z-value.
- Update the refresh buffer with the color (intensity) of the surface.

Step 6: Display the Image

After all surfaces are processed, the refresh buffer contains the final visible image, which is displayed on the screen.

Example

Suppose a cube is in front of a wall. At each pixel, the algorithm compares their depth values. Since the cube is closer, its color is stored in the refresh buffer, and the wall is hidden behind it.

B. Basic Functions of the Depth-Sorting Method (Painter's Algorithm)

The Depth-Sorting Method, also called the Painter's Algorithm, is another hidden surface removal technique. It works like a painter who paints the background first and the foreground last. As a result, nearer objects naturally cover farther objects.

Basic Functions

1. Sorting

First, all surfaces are sorted from the farthest to the nearest based on their depth (z-value).

2. Scan Conversion (Drawing)

After sorting, the surfaces are drawn one by one.

- The farthest surface is drawn first.
- The nearest surface is drawn last.

This allows the nearer surfaces to cover the farther ones.

3. Handle Overlapping Surfaces

If two surfaces overlap and their order is difficult to determine, the algorithm performs additional tests or changes their drawing order until the correct result is obtained.

Example

Imagine drawing a mountain, a tree, and a person:

1. Draw the mountain first (farthest).
2. Draw the tree next.
3. Draw the person last (nearest).

The person appears in front because it is drawn after the other objects.

6. Viewing Pipeline

The Viewing Pipeline is the sequence of steps that converts a 3D object into a 2D image so that it can be displayed on the computer screen. It transforms the object from its original coordinates to the final device (screen) coordinates.

Step 1: Modeling Coordinates

Create the object in its own local coordinate system.

↓

Step 2: Modeling Transformation

Translate, rotate and scale the object into world coordinates.

↓

Step 3: World Coordinates

All objects are placed in the complete scene.

↓

Step 4: Viewing and Projection Transformation

Position the virtual camera and project the 3D scene onto the viewing plane.

↓

Step 5: Projection Coordinates

Obtain the projected 2D coordinates.

↓

Step 6: Window-to-Viewport Transformation

Map the projected image into the display area.

↓

Step 7: Device Coordinates

Display the final image on the monitor.

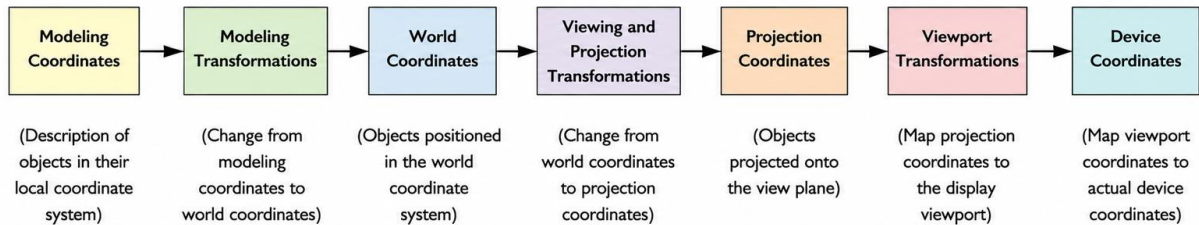


Figure 9-10

Pipeline for transforming a view of a world-coordinate scene to device coordinates.

Clipping Operations

Clipping is the process of removing the parts of an object that lie outside the viewing area (view volume) and keeping only the visible parts. In 3D graphics, clipping is performed against the boundary planes of the view volume instead of the boundary lines used in 2D.

Types of 3D Clipping Operations

1. Point and Line Clipping

In point and line clipping, the computer checks whether a **point or line** is inside or outside the view volume using the boundary plane equation:

$$Ax + By + Cz + D = 0$$

- If the result is less than 0, the point is inside.
- If the result is greater than 0, the point is outside.
- If a line crosses the boundary, the intersection point is calculated, and only the visible part is kept.

2. Polygon Clipping

Polygon clipping is used for 3D polygons. Each edge of the polygon is checked against the boundary planes, and the parts outside the view volume are removed.

Example:

If half of a rectangle is outside the viewing area, only the visible half is drawn.

3. Normalized View Volume

To make clipping easier, the view volume is first converted into a regular parallelepiped (or unit cube). This simplifies clipping because all boundary planes become parallel to the coordinate axes.

Example:

Instead of clipping against a complex shape, the computer clips against a simple box.

4. Region Codes

In 3D graphics, 6-bit region codes are used to quickly determine whether a point is:

- Left
- Right
- Above
- Below
- Front
- Back

These codes help the computer quickly accept or reject objects without performing unnecessary calculations.

5. Homogeneous Coordinate Clipping

Many graphics systems perform clipping using homogeneous coordinates, especially in perspective projection. This ensures that clipping remains correct after transformations such as shearing and scaling.

6. Hardware Implementation

Modern graphics cards perform clipping automatically in hardware using a graphics pipeline. This makes clipping much faster and improves rendering performance.

Example:

When playing a 3D game, objects outside the camera's view are clipped automatically by the graphics card, improving speed and performance.